

- [Análise do projeto ph.app.br](#)

Análise

Análise do projeto ph.app.br

- [Análise do projeto ph.app.br](#)
 - [Infraestrutura necessária](#)
 - [Definição do banco de dados Assistente_Virtual](#)
 - [Estudar como o site OpenAI pode ajudar a criar o *Assistant API da OpenAI*](#)
 - [Criar API de comunicação com o banco de dados](#)
 - [Banco de Dados de Informações Adicionais](#)
 - [Testes e Validação:](#)
 - [Documentação e Treinamento:](#)
 - [Manutenção e Atualização Contínua:](#)

Infraestrutura necessária

1. Criar máquina virtual *LinuxMint-ph* com tudo que precisamos para o projeto;

1. Criar usuário *phdemelo* e dar permissões de administrador; 
 1. adm
 2. cdrom
 3. dip
 4. lpadmin
 5. phpaulo
 6. plugdev
 7. sambashare
 8. sudo
 9. xrdp
2. Instalar *xrdp* e abrir para acesso publico para poder operar quando não estivermos próximo ao servidor. 
3. Ao tentar compartilhar para acesso publico tive que contratar um *IP fixo* *45.160.125.12* e contratar o plano *300 mega* da Smart Soluções; 
4. Instalar *PostgresSQL* e abrir para acesso publico; 
5. Criar o banco de dados *assistente_virtual*; 
 1. Criar documento: [Como trocar a senha do banco *postgreSQL* no linux caso se esqueça?](#)
6. Fazer backup do banco de dados *assistente_virtual*.
7. Instalar cliente de banco de dados *dbeaver-ce* para executar os script sql do projeto na criação e manutenção do banco de dados. 
8. Instalar *Servidor Web Apache2* e abrir para acesso publico para poder publicar os *Serviço Rest* para que a IA tenha acesso;

9. Instalar *IDE Lazarus* e o compilador *free pascal* para criar programas de cadastros e relatórios das tabelas do banco de dados. ✓
10. Criar *Serviço Rest* para comunicar-se com a inteligência artificial.
11. Instalar o interpretador *python* para estudo do Paulo Henrique; ✓
12. Compartilhar a pasta *user/local/Lazarus* para que eu possa usar na minha máquina a mesma versão do Lazarus que vou usar na máquina do Paulo Henrique. ✓

Definição do banco de dados Assistente_Virtual

1. operadores //Pode ser a secretária virtual
 1. id // tipo Integer serial
 2. nome // String com 50 posições não pode ser nulo
 3. login // String com 50 posições paulospacheco@yahoo.com.br
 4. password // String com 50 posições
 5. telefone : // String com 20 posições
2. hospitais
 1. id // tipo Integer serial
 2. nome // String com 50 posições pode ser nulo
 3. telefone // String com 20 posições pode ser nulo
3. status_da_agenda_ou_consulta
 1. id // tipo Integer serial
 2. nome // String com 20 posições pode ser nulo
4. médicos
 1. id // tipo Integer serial
 2. id_operadores //Tipo Integer
 3. nome // String com 50 posições pode ser nulo
 4. telefone // String com 25 posições pode ser nulo
 5. telefone_da_secretaria // String com 25 posições pode ser nulo
 6. login // String com 50 posições pode ser nulo
 7. se>nha // String com 20 posições pode ser nulo
5. serviço_de_agendas
 1. id // tipo Integer serial
 2. id_operador // tipo Integer
 3. nome // String com 100 posições pode ser nulo
 4. login // String com 50 posições pode ser nulo
 5. senha // String com 50 posições pode ser nulo
6. convênios
 1. id // tipo Integer serial
 2. id_médico // tipo Integer

3. nome // String com 50 posições pode ser nulo
4. login // String com 50 posições pode ser nulo
5. senha // String com 50 posições pode ser nulo

7. clientes

1. id // tipo Integer serial
2. nome // String com 50 posições. Obs: Não pode ser nulo
3. telefone_whatsApp; // String com 25 posições. Obs: pode ser nulo
4. e_mail // String com 50 posições e pode ser nulo
5. login // String com 50 posições e não pode ser nulo
6. senha // String com 20 posições e não pode ser nulo
7. id_convênio // tipo Integer pode ser nulo
8. dateTime_disponível // Tipo da data e pode ser nulo. Obs: Usado para pesquisar na agenda uma data mais próxima
9. matricula_no_convênio // String com 50 posições e pode ser nulo.

8. integração // Usado para ser logar

1. id // tipo Integer serial
2. endereço_do_site // String com 255 posições e não pode ser nulo
3. login // String com 50 posições e não pode ser nulo. Obs: Login para acessar o endereço_do_site
4. senha // String com 20 posições e não pode ser nulo. Obs: Senha para acessar o endereço_do_site
5. status // Campo lógico e não pode ser nulo Obs: Usado para saber se o site está conectado

9. expediente_do_medico_data // Chave múltipla id_medico+Data

1. id // tipo Integer serial
2. id_medico // tipo Integer e não pode ser nulo
3. dateTime // tipo data e não pode ser nulo

10. expediente_do_medico_horas // Chave múltipla id_medico+Data+Hora_inicial

1. id_expediente_do_medico_data // tipo Integer e não pode ser nulo
2. dateTime_inicial // Data e Hora inicial do expediente do médico e não pode ser nulo
3. dateTime_final // Data e Hora final do expediente do médico e não pode ser nulo

11. agenda

1. id // tipo Integer serial
2. id_Medico // tipo Integer e não pode ser nulo
3. id_Cliente // tipo Integer e não pode ser nulo
4. dateTime // Data e hora prevista da consulta e não pode ser nulo
5. dateTime_confirmação // Data e hora da confirmação da consulta e não pode ser nulo
6. id_convênio // tipo Integer e pode ser nulo caso id_formas_de_pagamento não for nulo.

7. id_status_da_agenda_ou_consulta //tipo Integer e não pode ser nulo
 8. id_formas_de_pagamento //tipo Integer e pode ser nulo caso id_convênio não for nulo.
 9. dateTime // Data e hora em que a agenda foi criada e não pode ser nulo
 10. observações // String com 255 posições e pode ser nulo
12. formas_de_pagamento
1. id // tipo Integer serialNatureza
 2. Nome // String com 30 posições e não pode ser nulo. Obs: Pode ser: 0 - Dinheiro; 1 - Pix; 2 - Convênio; 3 - Cartão de debito; 4 - Catão de credito Mastercard; 5 - Catão de credito Visa; etc....
13. consulta //Usado para baixar a agenda
1. id // tipo Integer serial
 2. id_agenda // tipo Integer e pode ser nulo. Obs: Uma consulta não é obrigado ter agenda.
 3. id_cliente // tipo Integer e não pode ser nulo.
 4. id_medico // tipo Integer e não pode ser nulo.
 5. dateTime // Data e hora da consulta e não pode ser nulo
 6. id_convênio // tipo Integer e pode ser nulo. Obs: Uma consulta não é obrigado ter convênio
 7. id_forma_de_pagamento // tipo Integer e pode ser nulo se id_convênio não for nulo.
 8. id_status_da_agenda_ou_consulta // tipo Integer e não pode ser nulo
 9. Observação // String com 255 posições e pode ser nulo

Estudar como o [site OpenAI](#) pode ajudar a criar o *Assistant API da OpenAI*

1. Checar se é possível comunicar-se com a IA usando o compilador free pascal; 
 1. Nota:
 1. Projeto *CHATGPT* criado por *Marcelo Maurin Martins* e está disponível no [github](#);
 2. Projeto *DelphiOpenAI* criado por HemulGM com licença *MIT*.
 1. [Readme do projeto](#)
 2. A IA precisa de *chave de autenticação* onde a mesma pode ser adquirida [aqui](#);
 3. A IA comunica-se com o programa por meio de prompts, usando a linguagem *JSON* para receber um pedido do usuário e a resposta é retornada com outro *JSON* com dados que o programa precisa para continuar o diálogo.
 1. Exemplo 01:
 1. Vamos criar algumas especificações de função para fazer interface com uma *API climática hipotética*.
 1. Passaremos essas especificações de função para a *API* de conclusões de bate-papo para gerar argumentos de função que

sigam a especificação.

```

tools = [
  {
    "type": "function",
    "function": {
      "name": "get_current_weather",
      "description": "Obtenha o clima atual",
      "parameters": {
        "type": "object",
        "properties": {
          "location": {
            "type": "string",
            "description": "A cidade e o
estado, por ex. São Francisco, Califórnia",
          },
          "format": {
            "type": "string",
            "enum": ["celsius",
"fahrenheit"],
            "description": "A unidade de
temperatura a ser usada. Inferir isso a partir da
localização dos usuários.",
          },
        },
        "required": ["location", "format"],
      },
    },
  },
  {
    "type": "function",
    "function": {
      "name": "get_n_day_weather_forecast",
      "description": "Obtenha uma previsão do
tempo para N dias",
      "parameters": {
        "type": "object",
        "properties": {
          "location": {
            "type": "string",
            "description": "A cidade e o
estado, por ex. São Francisco, Califórnia",
          },
          "format": {
            "type": "string",
            "enum": ["celsius",
"fahrenheit"],
            "description": "A unidade de
temperatura a ser usada. Inferir isso a partir da
localização dos usuários.",
          },
          "num_days": {
            "type": "integer",
            "description": "O número de

```

```
    "dias para previsão",  
    },  
    },  
    "required": ["location", "format",  
    "num_days"]  
    },  
    },  
    },  
    ],  
    ]
```

2. Exemplos:

1. Linguagem python:

1. [Como chamar funções com modelos de chat](#)
- 2.

4. Referências:

1. [api-reference](#)
2. [developer-quickstart](#)
3. [Introduction API OpenAI](#)
4. [Developer-quickstart](#)
5. [python-library](#)
6. [typescript-javascript-library](#)
7. [community.openai.com](#)
8. [step-1-setup-curl](#)
9. [Adding your API client to the Community Libraries page](#)
10. [Melhores práticas para engenharia imediata com API OpenAI](#)
11. [Referência da API - Solicitar corpo](#)
12. [Como contar tokens com tiktoken](#)
13. [DelphiOpenAI](#)
14. [incrível-chatgpt](#)

1. Obs:

1. Esse documento mostrou-me:
 1. Biblioteca [DelphiOpenAI](#) completa para comunicação com *chatgpt*,
 2. [Dicas sobre chatgpt-prompts](#)

15. .

Criar API de comunicação com o banco de dados

1. Tecnologias usadas no projeto:

1. Linguagem free pascal
 1. IDE Lazarus
 2. Pacotes
 1. LCL
 1. Para edição de formulários gráficos

2. fcl-web
 1. Para comunicação com protocolo HTTP
3. Maricarai
 1. Para criação de formulários LCL e HTML
4. CHATGPT-main
 1. Usado para comunicação com chatgpt
2. Linguagem javascript
 1. Editor VsCode
 1. Usado para editar o javascript que vou usar na IDE Lazaros
3. Objeto JSON
 1. Editor VsCode
 1. Usado para editar o JSON que vou usar na IDE Lazaros
4. Python
 1. Editor VsCode
 1. Só prendendo usar essa linguagem caso não consiga fazer com pascal.

2. Criar pasta *avm* onde *A=Assistente*, *V=Virtual* e *M=Médicos*

1. Criar pasta *avm/dm* onde *D=Data* e *M=module* ✓
 1. Nesta pasta deve ser registrada todo processamento que não dependa de componentes visuais;
 1. Criar datamodule *DM_Connections* para concentrar as conexões com o banco de dados;
 1. Este módulo deve conter os seguintes componentes com suas respectivas propriedades definidas abaixo:
 1. *SQLTransaction1* - Este componente é usado para definir o banco de dados a ser usado; ✓
 1. Name: 'SQLConnector1'; ✓
 2. ConnectorType : 'PostGresSQL'; ✓
 3. DataBaseName : 'assistente_virtual'; ✓
 1. Nota: O nome do database é sensível as letras maiúsculas e minúsculas. ✓
 4. HostName:'45.160.125.12'; ✓
 5. UserName: postgres; ✓
 6. Password:'masterkey'; ✓
 7. LoginPrompt : 'true'; ✓
 1. Nota : Usado para informar ao programa se o formulário pedindo o username e password deve ser executado;
 8. Transaction : 'SQLTransaction1'; ✓
 2. *SQLTransaction1* - Transação na qual um TSQLQuery é tratado;
 1. Name : 'SQLTransaction1';
 2. DataBase: 'SQLConnector1';

3. *SQLQuery1* - Classe para lidar com comandos SQL (com ou sem conjunto de resultados);

1. Name : *SQLQuery1*;

2.

2. Criar datamodule *DM_operador* para consultas para incluir, alterar e excluir a tabela de operadores;
3. Criar datamodule *DM_hospitais* para consultas para incluir, alterar e excluir a tabela de hospitais;
4. Criar datamodule *DM_status_da_agenda_ou_consulta* para consultas para incluir, alterar e excluir a tabela de *status_da_agenda_ou_consulta*;
5. Criar datamodule *DM_medicos* para consultas para incluir, alterar e excluir a tabela de médicos;
6. Criar datamodule *DM_serviço_de_agendas* para consultas para incluir, alterar e excluir a tabela de *serviço_de_agendas*;
7. Criar datamodule *DM_convenios* para consultas para incluir, alterar e excluir a tabela de convênios;
8. Criar datamodule *DM_clientes* para consultas para incluir, alterar e excluir a tabela de clientes;
9. Criar datamodule *DM_integracao* para consultas para incluir, alterar e excluir a tabela de integração ;
10. Criar datamodule *DM_expediente_do_medico_data* para consultas para incluir, alterar e excluir a tabela de *expediente_do_medico_data* ;
11. Criar datamodule *DM_expediente_do_medico_horas* para consultas para incluir, alterar e excluir a tabela de *expediente_do_medico_horas*;
12. Criar datamodule *DM_agenda* para consultas para incluir, alterar e excluir a tabela de agenda;
13. Criar datamodule *DM_formas_de_pagamento* para consultas para incluir, alterar e excluir a tabela de *formas_de_pagamento*;
14. Criar datamodule *DM_consulta* para consultas para incluir, alterar e excluir a tabela de consulta;
15. Criar datamodule *DM_Main* para concentrar todos os datamodule do projeto;

3. Criar pasta *lcl* onde *L=Lazarus*, *C=Component* e *L=Library*

1. Criar pasta *avm/lcl*
 1. Criar projeto de nome *avm_lcl*;
 2. Criar pasta *avm/lcl/units*

1. Nesta pasta deve ficar todos os formulários que dependem do pacote *lcl*
 1. Criar formulário *TForm_Main* para edição de todos os formulário do projeto;
 2. Criar formulário *TForm_model* para o modelo básico de um formulário *CRUD* que possa ser herdado;
 1. Adicionar os componentes: (obs: Checar se posso aproveitar o form básico que criei para o *gcic vcl.*)
 1. Class Panel;
 2. Class Menu;
 3. OnEventLogin;
 4. OnEvent????
 5. Criar menu de opções
 3. Criar formulário modelo básico de um formulário crud mestre/detalhe que possa ser herdado;
 1. Pensar....
 4. Criar formulário *_TForm_operado_r* edição da tabela de operadores;
 5. Criar formulário *TForm_hospitais* edição da tabela de hospitais;
 6. Criar formulário *TForm_status_da_agenda_ou_consulta* edição da tabela de *status_da_agenda_ou_consulta*;
 7. Criar formulário *TForm_medicos* edição da tabela de médicos;
 8. Criar formulário *TForm_serviço_de_agendas* edição da tabela de *serviço_de_agendas*;
 9. Criar formulário *TForm_convenios* edição da tabela de convênios;
 10. Criar formulário *TForm_clientes* edição da tabela de clientes;
 11. Criar formulário *TForm_integracao* edição da tabela de integração ;
 12. Criar formulário *TForm_expediente_do_medico_data* edição da tabela de *expediente_do_medico_data* ;
 13. Criar formulário *TForm_expediente_do_medico_horas* edição da tabela de *expediente_do_medico_horas*;
 14. Criar formulário *TForm_agenda* edição da tabela de agenda;
 15. Criar formulário *TForm_formas_de_pagamento* edição da tabela de *formas_de_pagamento*;
 16. Criar formulário *TForm_consulta* edição da tabela de consulta;

2.

4. Criar programa para coletar as informações do paciente

1. Obs?

1. Quais textos utilizar para interagir com o cliente no WhatsApp?

1. Podemos usar a IA para textos humanizados.
2. Outras forma é fazer perguntas e o cliente responde com o número da opção.

2. Cadastros necessários para comunicar-se com os clientes:

1. Medico
2. Convênio
3. Serviço_de_agendas
4. Integração
5. Expediente_do_medico_data
6. Expediente_do_medico_horas

3. Criar programa para confirmações de consultas.

1. Cadastros atualizados aqui:

1. clientes
2. Agenda
3. Disponibilidade_do_Paciente

4. **Confirmação de Consultas:** 1. Utilize a *API do WhatsApp* para enviar 2. Implementar a *API de voz da OpenAI* para fazer *chamadas de voz automatizadas* para *confirmação de consultas*.

Banco de Dados de Informações Adicionais

1. Criar um *banco de dados adicional* para armazenar informações sobre *convênios*, *preços de consultas* e *hospitais onde o médico atende*.

Testes e Validação:

1. Realizar *testes abrangentes* para garantir o funcionamento adequado de todas as funcionalidades.

Documentação e Treinamento:

1. Preparar *documentação detalhada* e *oferecer treinamento* para o *médico* e a *equipe* sobre o uso da *secretária virtual*.

Manutenção e Atualização Contínua:

1. Criar um *plano de manutenção e atualização* para manter a segurança, eficiência e compatibilidade do sistema.

